

Quickstart

The modules provided by the Python DS-API package provide convenient utilities that allow interacting with Pulse and performing common data science tasks. This is a list of the available libraries:

- `ds_api` . A pythonic interface for Pulse using `pyjnius`. This library is the entry point to interacting programmatically with Pulse. It provides tools that connect to Pulse and perform common data science operations, such as accessing data sources, running plans, training and evaluating models, etc.
- `ds_api.exploration` . Utilities for efficient exploratory data analysis via PySpark. You can efficiently profile your data, easily generate detailed reports, and quickly access personalized results of the exploration in the form of statistics, validations and visualizations.
- `ds_api.sampling` . Efficient sampling utilities through PySpark. It provides a stratified and a random sampling strategy, but can be easily extended to user defined strategies. It supports smart optimized sampling that preserves the minimal history of entities, so historical profiles can be correctly computed while maximizing the sampling reduction power.

Recipes

The best learning resource to get familiar with the DS-API and its available libraries are the notebooks provided in the JupyterLab environment in Pulse.

DS-API

The entry point to the DS-API and its functionalities is to connect to the Pulse server and instantiate a `Feedzai` object:

```
from ds_api import Feedzai, Server

PULSE_SERVER_ADDRESS = "localhost"
PULSE_SERVER_PORT = 1099
# Usual pulse.manager.rmi.localregistryport port. Confirm this in Admin UI, inside Pulse configuration.
pulse = Server.connect(address=PULSE_SERVER_ADDRESS, port=PULSE_SERVER_PORT)
feedzai = Feedzai(pulse=pulse)
```

Now you can connect to a Pulse application, and for instance list its available data sources:

```
app = feedzai.app(name="app-name")
app.pullDataSources()
# ["data source1", "data source2"]
```

To analyze a datasource:

```
DATASOURCE = "data source1"
datasource = app.pullDataSourceByDesc(DATASOURCE)
datasource.analyze()
```

To convert a Pulse datasource to a PySpark DataFrame:

```
spark_session = feedzai.get_spark_session()
data = feedzai.pulse_to_pyspark(spark_session, app, DATASOURCE)
```

More detailed demonstrations on how to use the DS-API can be found in the following notebooks that are available in the JupyterLab integration with Pulse:

- **01_getting_started.ipynb** - shows the first steps of using the DS-API: loading data from/to Pulse and converting data to a format that you can use in JupyterLab.

- Demonstration of how to interact with Pulse's data sources, plans and models using the DS-API:
 - **01_datasources.ipynb**
 - **02_plans.ipynb**
 - **03_models.ipynb**
 - **04_data_loading.ipynb** - shows how to use the DS-API to programmatically load data into Pulse.
-

Exploration

Continuing from the example in the previous section. To profile a PySpark DataFrame for exploration:

```
from ds_api.exploration import compute_stats

TIMESTAMP_FIELD = "timestamp"
COST_FIELD = "amount"
TARGET_FIELD = "fraud"
TARGET_VALUE = "true"
TARGET = (TARGET_FIELD, TARGET_VALUE)

results = compute_stats(
    data=data,
    timestamp_field=TIMESTAMP_FIELD,
    target=TARGET,
    cost_field=COST_FIELD
)
```

Now you can display the exploration report (in JupyterLab) and save it:

```
from IPython.core.display import HTML

HTML(results.report)
results.save_report_html("report.html")
```

You can also quickly access statistics and visualizations using a convenient API. Some examples:

```
results.time_dependencies
results[COST_FIELD].quantile()
results[TARGET_FIELD].frequencies()
results[TIMESTAMP_FIELD].plot_with_fraud_rate()
```

The notebook **05_data_exploration.ipynb**, available in Pulse's JupyterLab environment, provides a more exhaustive guide to the exploration library.

Sampling

Continuing from the examples in the previous sections. First initialize the desired sampler:

```
sampler = StratifiedRandomUndersampler(
    target_field=TARGET_FIELD,
    target_value=TARGET_VALUE,
    balance=0.9,
)
```

Create a filter if you want filter the events that will be marked by the sampling strategy:

```
from pyspark.sql import functions as F

EVENT_TYPE_FIELD = "event_type_field"
SCORABLE_EVENTS = ["scorable_event_type1", "scorable_event_type2"]
scorable_events_filter = F.col(EVENT_TYPE_FIELD).isin(SCORABLE_EVENTS)
```

Now you can sample the PySpark DataFrame. In this example the minimal history of the given entity field is preserved in the output dataset, so historical profiles can correctly be computed:

```
ENTITY_FIELD = "card_id"
MAX_WINDOW_SIZE = 30 # days

sampled_data = sampler.sample(
    train_data,
    filter_expr=scorable_events_filter,
    entity_field=ENTITY_FIELD,
    timestamp_field=TIMESTAMP_FIELD,
    recovery_days=MAX_WINDOW_SIZE
)
```

A more detailed illustration on how to use the sampling tools for model training and testing purposes is available in Pulse's JupyterLab environment via the notebook **06_data_sampling.ipynb**.